USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Milestone 2 Verification Report

UM-NATOS-009

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-009	1.3 · 2026-08-15	**PASS** — §11.4 added, ageing bounds the starvation strict priority allows	Internal

1. Abstract

Milestone 2 establishes preemptive multitasking: several independent execution contexts share one core, each suspended at an arbitrary instruction by a timer interrupt and later resumed as though nothing happened.

M1 proved that the kernel could be interrupted and resume *itself* correctly. M2 is the harder claim, because the interrupt now returns to a **different** context than the one it interrupted. Everything above this layer — drivers, the VM host, any notion of concurrency — assumes it works.

The milestone passes. Section 6 documents the defect that dominated the effort, including two wrong diagnoses and one wrong fix that was kept anyway, because the reasoning is more reusable than the result: the same hazard applies to every C function this kernel will ever call from an interrupt handler.

2. What M2 had to establish

Claim	How it fails if untrue
A task can be created that has never run	The first switch jumps to a fabricated frame and lands nowhere
A running task can be suspended mid-instruction-stream	Registers or PC are lost; corruption appears later, elsewhere
A suspended task can be resumed exactly	Silent data corruption in the resumed task
The scheduler distributes time	One task starves the others; looks like a hang
Tasks do not corrupt each other's stacks	Overflow into a neighbour; no MMU to catch it

The third and fourth are the ones that actually broke.

3. Design decisions

3.1 One way for a task to come into existence

An earlier design adopted the boot context as task 0, capturing its stack pointer on the first interrupt, and fabricated frames for every other task. That is two mechanisms for the same thing, and only one of them

worked: tasks 1 and 2 ran, task 0 never resumed.

The design was changed so that **every task without exception is created by fabricating a frame that looks as though it had already been interrupted.** `kmain` creates the tasks, starts the tick, and is then abandoned — its stack is never reclaimed and it never runs again. `g_current` starts at `-1`, which tells the scheduler there is no outgoing context to save on the first switch.

This deleted the failing path rather than debugging it, and left one code path to be correct about instead of two.

3.2 The switch happens inside the interrupt, not beside it

There is no separate `switch_to()` routine. The level-3 handler saves the full context onto the interrupted task's own stack, passes the stack pointer to `task_schedule()`, and resumes on whatever pointer comes back. If that pointer belongs to a different task, the return *is* the context switch.

`task_yield()` therefore does not switch directly — it pulls the comparator deadline forward so the ordinary tick fires almost immediately. One switching mechanism, exercised constantly, rather than a second one exercised rarely.

3.3 EPC3 and EPS3 must travel in the frame

`RFI 3` restores PC and PS from `EPC3` / `EPS3`. These are **single registers, not per-task state.** Swapping stack pointers alone would resume the new task's registers at the *old* task's program counter. Saving both into each frame and restoring them before `RFI` is what converts a stack swap into a task switch.

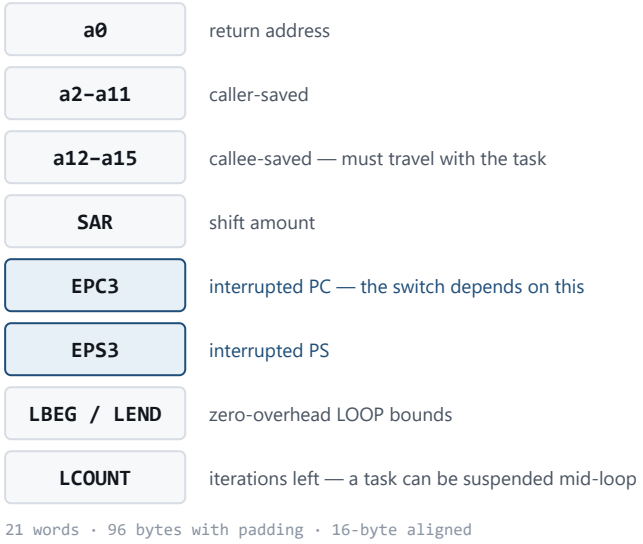

Figure 1. *Saved-context frame. EPC3/EPS3 are what make a stack swap into an actual task switch.*

3.4 Round robin, and nothing cleverer

Selection scans forward from the current task and takes the first `READY` one. No priorities, no sleeping, no blocking — none of those have a consumer yet, and each would be an untested mechanism in the one code path that must be correct.

> All three exist now; see §11. Each arrived when something needed it — a mutex needed blocking, priorities needed sleeping — which is the order this section was holding out for.

4. Implementation

4.1 Frame layout

96 bytes, 21 words, 16-byte aligned as Xtensa requires.

Offset	Contents	Why
0	`a0`	Return address; clobbered by `call0` and saved before that happens
4–56	`a2` – `a15`	Both caller- and callee-saved sets: the resumed task expects *its* values
60	`SAR`	Clobbered by any shift
64	`EPC3`	Interrupted PC
68	`EPS3`	Interrupted PS
72–80	`LBEG` , `LEND` , `LCOUNT`	Zero-overhead loop state — see §6.5

`a1` is not stored: the frame's own address *is* the saved stack pointer.

4.2 Handler sequence

```
addi a1, a1, -96        ; frame on the interrupted task's stack
save a0, a2..a15, SAR, EPC3, EPS3, LBEG, LEND, LCOUNT
clear PS.EXCM           ; MUST precede any C — see §6.3
call0 timer_isr         ; tick bookkeeping, comparator re-arm
mov  a2, a1
call0 task_schedule     ; a2 in = current sp, a2 out = sp to resume
mov  a1, a2             ; from here a1 may be a different task's stack
restore EPS3, EPC3, SAR, LBEG, LEND, LCOUNT, a0, a2..a15
addi a1, a1, 96
rfi  3
```

Loop state is saved **before** any C is called, because `task_schedule` contains a loop of its own and would otherwise destroy what it was meant to preserve. `LCOUNT` is restored **last** of the three, since it is the register that arms the hardware loop and the bounds must already be valid when it becomes live.

4.3 Task creation

`task_create` fills a stack with `0xEEEEEEEE` , writes a `0x57ACC0DE` guard at the lowest word, and builds a frame at the top with all registers zero, `EPC3 = entry` , and `EPS3` taken from the running kernel with `INTLEVEL` forced to 0. Inheriting PS rather than fabricating one keeps the task in the same execution mode as its creator instead of a guessed one.

5. Verification method

Three tasks, all created identically:

- **worker-a**, **worker-b** — hold four values live across a compiler barrier positioned where an interrupt can land, then verify two of them still equal their seed and its complement. Any register or stack fault increments a corruption counter. Registers are deliberately *not* pinned with explicit `__asm__("aN")` bindings: that claims registers the compiler may already be using, and the writes then land on arbitrary memory. An earlier build did exactly this and manufactured a bug that did not exist.
- **report** — a task like any other, suspended and resumed on the same schedule. That its output stays coherent is itself part of the test.

Instrumentation added during debugging and deliberately retained: entry markers per task, a dump of the frame about to be restored, the task table, and per-candidate scheduler probes behind `TRACE_PROBES`.

6. The defect

6.1 Symptom

Tasks entered correctly and then the board went silent. Switch 4 reported `2 -> 2` — the scheduler selecting the task it was already running — and repeated forever. The task table printed microseconds later showed all three tasks `st=1 (READY)` with stable stack pointers. The scheduler was refusing candidates that were plainly available.

Ticks 1–3 enter frames built by task_create. Tick 4 is the first resume of a frame the handler saved — the only mechanism that had never executed successfully, and where the defect surfaced.

Figure 2. *The first five context switches on hardware. Entering a fabricated frame and resuming a saved one are different mechanisms.*

6.2 Two earlier conclusions that were wrong

Both were recorded in the repository before being disproved, and are corrected here rather than quietly dropped.

"The timer interrupt is not firing." It was firing. `ticks=0` was sampled before the first tick had occurred, because the reporter waited 100 ticks before printing and the diagnostic loop printed faster than the tick period.

"The board goes silent, so the switch is fatal." The switch worked. The silence was the workers doing exactly what they were written to do: spin without producing output. Every failure mode in this kernel — masked interrupt, bad `RFI`, dead task, working-but-quiet task — presents identically as nothing on the wire. One entry marker per task turned that silence into a sequence and settled it immediately. That should have been the first instrument, not the fourth.

6.3 Root cause — `PS.EXCM` disables the zero-overhead loop

GCC compiled the round robin into an Xtensa **zero-overhead** `LOOP`:

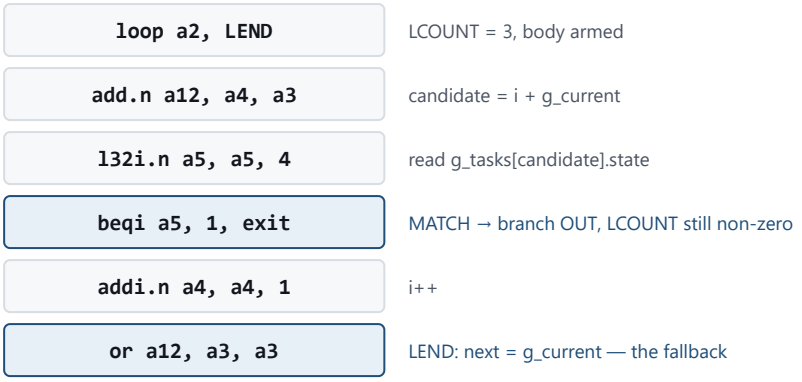

Figure 3. *The scheduler's round robin as GCC compiled it. The no-match fallback occupies the LEND slot, which is reached on every call while PS.EXCM is set.*

On Xtensa, the zero-overhead loop-back is disabled while `PS.EXCM` is set. The `LEND` comparison never fires, so the body executes exactly once and falls straight through into whatever instruction occupies the `LEND` slot. Here that instruction is `or a12, a3, a3` — the `next = g_current` fallback.

Hardware sets `EXCM` on interrupt entry. The handler had never cleared it, so every call into C ran with hardware loops silently degraded to a single iteration.

This accounts for the whole observation set, including the parts that looked arbitrary:

Observation	Explanation
Switches 1–3 correct	All were first-iteration hits — one iteration is enough
Switch 4 wrong	First case needing a second iteration; task 3 is `UNUSED`
Result was always `g_current`	That is precisely the instruction at `LEND`
Instrumentation fixed it	A body containing a call cannot be a hardware loop
`volatile` counter fixed it	Denies GCC the constant trip count
Saving `LCOUNT` did nothing	`LCOUNT` was never the mechanism

The scope is far wider than the scheduler. GCC emits hardware loops for ordinary counted C loops, so *every* C function reachable from an interrupt handler was affected — every future driver, and the VM interpreter. The scheduler is simply where it happened to become visible, and it became visible only because a single wrong iteration still produced a plausible-looking answer three times in a row.

6.4 The decisive experiment

The hypotheses were separated with a test hook, `task_select_probe()`, holding the selection loop in its original hardware-loop form. Calling the **same function** from two contexts in the **same build**:

```
from kmain (PS.EXCM = 0):   probe(2) = 0    correct
from the ISR (PS.EXCM = 1): probe(2) = 2    wrong
```

with the handler's own PS dumped alongside: `isr ps=0x00060733` — `INTLEVEL=3`, and bit 4 `EXCM` **set**. After the fix, the same line reads `isr ps=0x00060723` with `EXCM` clear and `probe(2) = 0`.

This is what a controlled comparison is for. The earlier A/B in §6.5 varied the *code generation* and could only ever show correlation; varying the *context* while holding the code identical isolates the cause.

6.5 A hypothesis that measurement rejected

The first explanation was stale `LCOUNT`: a task suspended mid-loop, its loop state lost across the switch, the hardware later branching back to a previous `LEND`. The context frame was extended to save `LBEG` / `LEND` / `LCOUNT`, and the build reflashed.

The symptom did not change, and every dumped frame showed `lct=0x00000000`. No task had ever been suspended mid-loop.

The frame change was nevertheless **kept**, on its own merits: a task genuinely can be suspended mid-loop, and losing that state would corrupt it on resume. ESP-IDF's context switch saves these three registers for the same reason. A real fix for a real hazard — just not for this defect. Recorded because a confident prediction that measurement rejected is the most reusable part of the exercise.

6.6 Fix

`_handler_level3` clears `PS.EXCM` after saving the context and before calling any C:

```
rsr.ps   a2
movi     a3, ~0x10
and      a2, a2, a3
wsr.ps   a2
rsync
```

Safe at this point: `PS.INTLEVEL` is still 3, so interrupts remain masked, and `EPC3` / `EPS3` are already saved, so the eventual `RFI` is unaffected. It also means a fault inside the handler now reaches the kernel exception vector and the panic handler rather than the double-exception vector.

The `volatile` workaround has been **removed**. The selection loop is a plain counted loop again, compiles to a hardware `LOOP`, and is correct. Verified at 3,418 ticks and 1,140/1,139/1,139 switches with zero corruption.

7. Results

Sustained run, 10 ms tick, three tasks:

```
t=202   switches r/a/b=68/67/67     work a/b=1528960/1698281    guards=ok  corrupt=0
t=403   switches r/a/b=135/134/134  work a/b=3104989/3426830    guards=ok  corrupt=0
t=604   switches r/a/b=202/201/201  work a/b=4681019/5155380    guards=ok  corrupt=0
t=805   switches r/a/b=269/268/268  work a/b=6257049/6883930    guards=ok  corrupt=0
t=1006  switches r/a/b=336/335/335  work a/b=7833079/8612480    guards=ok  corrupt=0
t=1207  switches r/a/b=403/402/402  work a/b=9409109/10341029   guards=ok  corrupt=0
```

Distribution. 403/402/402 across three tasks — even to within one switch, which is the expected residue of where the run was sampled.

Integrity. `corrupt=0` across ~1,200 preemptions. Both workers held their invariants across every suspension.

Stacks. 463 of 512 words free in both workers, stable across the run: about 196 bytes used, no growth. Guards intact.

Switch cost. 96 bytes of stack and 21 register transfers per switch, twice (save and restore).

8. Watchdog — corrected from inference to measurement

UM-NATOS-006 and UM-NATOS-008 recorded the watchdog state as "inference, not measurement", reasoning that survival through the capture window implied nothing was armed. **That inference was wrong.** The bootloader arms the RTC watchdog and expects the application to take ownership. M0 and M1 survived by luck of timing.

It has now been measured: `RTCWDT_RTC_RESET` on every boot once M2 kept the CPU busy, and `WDTCONFIG0` reading back `0x0` after the disable. The RTC watchdog and both timer-group watchdogs are now disabled explicitly at kernel entry, behind the `0x50D83AA1` write-protect key, and re-locked afterward.

They are disabled rather than fed, because a watchdog is only useful once something is responsible for feeding it. Until the scheduler owns that duty, an armed watchdog reboots working code. Re-enabling belongs in M3 with an idle task feeding it, at which point it becomes a genuine hang detector.

9. What M2 does not establish

- **No memory protection.** The ESP32 has no MMU paging. A native task can corrupt any other; guards catch overflow, nothing catches a wild pointer. Isolation arrives only with the bytecode VM (UM-NATOS-001 §4.2).
- ~~No blocking.~~ Superseded by §11: `TASK_BLOCKED` and `TASK_SLEEPING` both exist. What remains missing is any wait other than a lock or a deadline — there is no way to wait for an event.
- ~~No priorities.~~ Superseded by §11. What remains is that LOW starves absolutely: there is no aging and no anti-starvation of any kind.
- **Fixed ceiling.** `TASK_MAX = 4`, 2 KB stacks, statically allocated.
- ~~No idle task.~~ Added with blocking, and it executes `WAITI` rather than spinning.
- **Single core.** APP_CPU is untouched.
- **No audit of other `EXCM` consequences.** §6.3 establishes that hardware loops were degraded while `EXCM` was set, and clearing it fixes that. Whether anything else in the kernel silently depended on `EXCM` being set for the duration of the handler has not been examined. Nothing suggests it does — the handler is short and the only C it calls is the tick and the scheduler — but the question was not asked systematically.

10. Metrics

Quantity	Value
Image size	5,312 B (includes retained instrumentation)
Frame	96 B / 21 words
Tick interval	800,000 cycles (~10 ms)
Tasks	3 of 4 slots
Stack per task	2 KB; ~196 B peak use
Ticks observed	3,418
Switches observed	1,140 / 1,139 / 1,139
Corruption events	0
Build cycles spent on §6	12
Wrong hypotheses recorded	3
Instructions in the fix	5
Priority levels	3, plus ageing credit up to 3
Ageing threshold	30 ticks per level, capped at 3
Worst-case wait for a ready task	~600 ms, bounded
Longest wait observed under stress	35 ticks (350 ms)
Ageing rescues at shipped settings	0 — inert, as intended for a backstop

11. The scheduler since M2

M2 shipped a strict round robin over ready tasks. Four things have been added, and the order matters: each was forced by the one before it, and the fourth was forced by a failure the third made possible.

11.1 Blocking, then sleeping, then priorities

`TASK_BLOCKED` arrived with the mutex (UM-NATOS-014 §3). A blocked task is skipped by the scheduler, which also required an idle task — without one, a moment where every task is waiting has nothing to switch to, and the old fallback of resuming the interrupted task would have run a task that had just blocked.

`TASK_SLEEPING` arrived with priorities, and is what makes them usable. A sleeping task carries a tick deadline; the scheduler wakes expired sleepers on every tick, so a sleep resolves to one tick.

Priorities are three levels, strictly ordered, round-robin among equals. The implementation is one loop: the scan starts past the current task and takes a candidate only on *strictly* greater priority, which yields both properties at once — among equals the first one met wins, and the scan begins somewhere different each time.

11.2 Strict priority is only safe because tasks sleep

A high-priority task that spins starves everything beneath it absolutely. This kernel's own `while (timer_ticks() < until) task_yield();` idiom would have done exactly that, and every such wait is now a sleep.

That is not a hypothetical. The demonstration workers were first placed at LOW and performed **exactly zero iterations** — the NORMAL band never empties, because the application host never sleeps. The failure was quiet in the worst way: `corrupt=0` continued to be reported, and means nothing when no work is being done. An instrument reading healthy because its subject had stopped is the same shape as §6 and as UM-NATOS-017 §6.

LOW starves absolutely and there is no aging. Anything placed there must be genuinely discardable.

> *Revision 1.3: the second sentence of that claim is no longer true.* Ageing was added after strict priority starved a task to complete silence — see §11.4. The first sentence still holds as a design intent: LOW means "run only when nothing else wants the CPU", and ageing bounds the wait rather than abolishing the ordering.

11.3 Measured

```
renderer alone at HIGH:     2 fps -> 8.5 fps
with a busy NORMAL band:    ~3 fps
workers: 821,248 iterations each, exactly equal, corrupt=0
```

> *Revision 1.3: these frame rates were computed as frames per TICK, and the tick was later found to stall for up to 183 ms at a time (UM-NATOS-008 §8). The comparison is still valid — both figures came from the same clock — but the absolute values are not trustworthy. The renderer measured 16.0 fps once the clock was corrected.*

Everything except the renderer shares NORMAL: the speedup comes from the renderer being HIGH, not from anything being LOW.

The workers additionally sleep one tick per 2,048 iterations. They exist to prove the switch preserves registers across arbitrary suspension, which needs them running *continuously*, not *constantly*. Spinning flat out bought no extra evidence, cost the renderer half its frame rate, and — because it ended the mutex thrash between them — sleeping raised their own throughput almost tenfold.

11.4 Ageing, because sleeping was not enough

Revision 1.3.

§11.2 argues that strict priority is safe **because tasks sleep**. That argument is sound and it is not a guarantee — it depends on every high-priority task being written to yield often enough, and nothing enforces it.

It failed the first time a task was made hungrier. Shortening the display task's sleep from 8 ticks to 2 left it ready roughly 61% of the time, and the reporter task stopped being scheduled at all:

```
reporter lines in 20 s     0
crash                      none
watchdog reset             none
```

Nothing was wrong that any existing mechanism could see. **The hang detector asks whether ANY distinct switch happened, not whether every ready task got a turn** (UM-NATOS-019 §2), and switches were happening constantly between the display task and its neighbours. The only reason the starvation was noticed at all is that the starved task happened to be the one that prints. A quieter victim would have produced no symptom.

The policy

A ready task gains effective priority the longer it is passed over:

```
effective = base + min(waiting / TASK_AGE_TICKS, TASK_AGE_MAX)
```

Three properties are deliberate:

- **Ageing is a property of the SELECTION, not of the task.** The base priority is never modified, so a task that finally runs returns to its declared priority automatically. That is what keeps this distinct from priority inheritance (UM-NATOS-014 §9), where the priority genuinely changes and genuinely has to be undone. A mechanism that must remember to restore something is a mechanism that can forget.
- **Only READY tasks earn credit.** A task waiting on a deadline or a mutex is not being treated unfairly by the scheduler; crediting it would let it barge ahead the moment it became runnable.
- **The bound is a latency, not a throughput guarantee.** `TASK_AGE_TICKS × 2` is 600 ms at the shipped values — a ready task reaches the front within that regardless of what sits above it. A task that is aged in and then immediately blocks still makes no progress, and that is the caller's problem.

Measured

Re-running the configuration that caused the starvation:

```
reporter lines in 20 s     0  ->  8
fair maxwait               35 ticks (350 ms), bounded as designed
fair rescues               114 decisions changed by ageing
```

At the shipped sleep value it reads `rescues=0, maxwait=15` : **inert in normal operation**, which is correct for a backstop. Both numbers are reported anyway, because a fairness policy nobody measures is a fairness policy nobody has, and `rescues=0` is the only thing that distinguishes "never needed" from "never working".

What it did not do

It did not raise the frame rate. With ageing in place *and* the display sleep shortened, the renderer still ran at 3.0 fps — identical to before. The frame rate was bounded by lock contention (UM-NATOS-014 §10), which is a different problem that ageing does not touch.

Ageing fixed a class of silent failure. It is not a performance feature and is not recorded as one.

12. References

- UM-NATOS-001 §4.2 — isolation model and why native tasks are not applications
- UM-NATOS-003 — `call0` ABI selection; why register windows are absent
- UM-NATOS-006 §6, UM-NATOS-008 §6 — watchdog notes corrected by §8 above
- UM-NATOS-008 — M1 interrupt dispatch and register-integrity measurement

- `kernel/vectors.S` — handler, frame layout, and the `EXCM` clear of §6.6
- `kernel/task.c` — scheduler, fabricated frames, and `task_select_probe()`
- `kernel/watchdog.c` — disable sequence and register addresses